

提升法

前面我们讲解了“随机森林”算法，除了该方法外，本章介绍的提升法是另外一种集成学习的方法，与随机森林或袋装法最大的不同是，各个基学习器是序列集成的，也就是说基学习器之间是有顺序的，完成上一个基学习器之后才能开展下一个，在每一轮的训练中，下一个基学习器都能通过权重更新的方式较上一个基学习器实现提升。本章我们讲解提升法的基本原理，并结合具体实例讲解该算法在python中解决分类问题和回归问题的实现与应用。

目录

C O N T E N T S

- 15.1 提升法的基本原理
- 15.2 数据准备
- 15.3 回归提升法示例
- 15.4 二分类提升法示例
- 15.5 多分类提升法示例
- 15.6 习 题



PART 01

提升法的基本原理

提升法的概念与原理

如果个体学习器之间存在强依赖关系、必须串行集成，则应选择提升法（Boosting方法）。提升法的主要代表包括AdaBoost、GBDT、XGBoost及其改进等。各类Boosting方法的基本思想是一致的，典型的操作实现过程是：

- 1、首先从训练集用初始权重训练出一个基学习器 x_1 ，得到该学习器的经验误差；然后对经验误差进行分析，基于分析结果调高学习误差率高的训练示例的权重，使得这些误差率高的训练示例在后面的学习器中能够受到更多的关注；再后基于更新权重后的训练集训练出一个新的弱学习器 x_2 。

提升法的概念与原理

2、不断重复这一过程，直到满足训练停止条件（比如基学习器达到指定数目），生成最终的强学习器。注意：在训练的每一轮都要检查当前生成的基学习器是否满足基本条件。

3、将所有基学习器预测结果进行加权融合并输出，比如AdaBoost通过加权多数表决的方式，即增大错误率小的基学习器的权值，同时减小错误率较大的基学习器的权值。

综上所述，提升法在单个基学习器训练过程中，通过提高训练错误示例权重，以使其受到更多关注，从而在下一轮训练中得到提升；而最终模型中则提高错误率小的基学习器的权值，从而实现整体预测能力的提升。

|| AdaBoost (自适应提升法)

一、分类问题的提升法

AdaBoost最初仅适用于分类问题，是基学习器的加权线性组合，其基本思想是：首先根据训练集的大小，按照均匀分布原则，初始化样本示例权重，也就是说所有样本示例的权重都相等；然后在后面的每一轮训练中，都依据样本示例预测表现进行权重更新，如果样本被错误分类，则其权重会被乘以一个大于1的倍数，如果样本被正确分类则保持原权重不变，然后通过权重标准化（所有样本示例权重之和为1）的方式，提升被错误分类样本示例的权重，使其在新一轮训练中得到更多的关注，直至该样本被正确分类。在满足一定规则时（达到规定迭代次数或规定误差率），训练停止；训练过程中，每一轮都会得到一个基学习器，针对分类越为准确的基学习器给予一个越高的权重，然后加权形成最终的强学习器。

AdaBoost (自适应提升法)

不难看出，AdaBoost算法一方面不断进行权重更新，使得模型能够更加关注在特征空间内预测错误的区域，从而降低了整体的偏差；另一方面，该算法也是多个基学习器的集成，同样有样本搅动，所以也在一定程度上降低了整体的方差。与我们上一章中介绍的Bagging相比，Bagging方法采用的是Bootstrap（有放回的均匀抽样），训练样本集的选择是随机的，各轮训练样本集之间相互独立；而AdaBoost则根据错误率来取样，从加权的分布中进行抽样，各轮训练集的选择与前面各轮的学习结果有关，因此通常来说AdaBoost的分类精度要优于Bagging。但同样的，Adaboost算法会使得难于分类样本的权值呈指数增长，后续的训练过程将会过于偏向这类困难样本，从而导致Adaboost算法容易受极端值干扰。

从统计学的角度来理解，二分类问题的AdaBoost算法，等价于使用指数损失函数的前向分段加法模型（证明过程略，可参阅相关文献）。

|| AdaBoost (自适应提升法)

二、回归问题的提升法

而针对回归问题，也有回归问题的提升法（或 L_2 提升法），本质上是使用误差平方损失函数的前向分段加法模型，将残差作为响应变量，对特征变量进行回归。使用的基学习器可以是线性回归算法，也可以是单颗回归决策树，如果是回归树时，需要特别考虑决策树的数量（ m ）、决策树的分裂次数（ d ，也称交互深度）以及学习率（ η ，也称收缩参数）等超参数。

关于决策树的数量（ m ）：在 Bagging 方法中，由于随机森林是使用 m 颗决策树的预测结果进行平均或按照多数票规则确定，所以根据大数据定律，增加 m 不会导致过拟合，我们选择最优 m 的目的往往是在于保证其足够大，能够充分降低袋外误差， m 到了一定数值后在降低袋外误差方面的贡献微乎其微，但也不会产生过拟合的反面作用；而在 AdaBoost 算法中这一情况则显著不同，因为每一轮的训练都倾向于降低基学习器的偏差，从而越来越多的融合了训练样本的个体特征，所以需要特别注意不能因为取值过大而产生过拟合现象。具体 m 的取值可以通过交叉验证方法来寻找。

AdaBoost (自适应提升法)

关于决策树的分裂次数 (d)：前面我们介绍决策树的时候，不难发现决策树采用的特征变量之间是有交互效应的，比如“工作年限较短且债务收入比较高的客户违约率很高”，其中就包含了“工作年限”和“债务收入比”两个特征变量之间的交互效应。不难发现，决策树分裂多少次，就包含多少个特征变量之间的交互效应，交互效应的层次根据实际需要来确定，通常情况下，选择 4~8 即可。

关于决策树的学习率 (η)： η 的取值范围是 $(0, 1)$ 。学习率用来限制学习的速度， η 的取值越小，学习的速度越慢，每一轮的训练在降低基学习器的偏差进步方面就越小。所以如果 η 设置的比较小，那么就能够更好的避免过拟合。 η 需要和决策树的数量 (m) 配合使用，决策树的数量越多，也就是训练开展的轮数就越多，配套较低的 η 可能是合适的；但是如果 m 取值较小， η 取值也较小的话，就会走向另一个极端，可能出现欠拟合。通常情况下，选择 0.1 或 0.01 即可。

|| 梯度提升法 (Gradient Boosting machine)

前面我们讲述的AdaBoost算法，本质上是使用指数损失函数（二分类问题）或误差平方损失函数（回归问题）的前向分段加法模型实现算法优化。但是如果我们把损失函数扩展到一般损失函数，这一实现过程很可能就会变得非常复杂。为了解决这一问题，Freidman提出了梯度提升法，梯度提升法是AdaBoost算法的推广，其特色在于以非参数方式估计基函数，并在函数空间内使用梯度下降（Gradient Descent）进行近似求解。所谓梯度下降，就是使损失函数越来越小，在求解约束优化问题时，梯度下降是最常采用的方法之一。具体来说，就是根据当前模型损失函数的负梯度信息来训练新加入的基学习器，然后将训练好的基学习器以累加的形式结合到现有模型中，通过梯度提升的方式，不断优化参数，直至达到局部最优解。

|| 梯度提升法 (Gradient Boosting machine)

损失函数的负梯度数学公式为：

$$-g(x) = - \left\{ \frac{\partial L(y, F(x))}{\partial F(x)} \right\}$$

在微积分里面，从数学公式上看，对多元函数的参数求偏导，然后把求得的各个参数的偏导以向量的形式写出来，就是梯度。负梯度就是梯度的负值，所以损失函数的负梯度其实就是损失函数针对参数求偏导的负数。从几何的角度理解，梯度就是函数变化最快的地方，沿着负梯度向量方向梯度减少最快，从而更容易找到函数的最小值；同样的，沿着梯度向量方向梯度增加最快，从而更容易找到函数的最大值。

在梯度提升法中，我们需要最小化损失函数，也就是要找到损失函数的负梯度，沿着负梯度向量方向一步步的迭代求解，得到最小化的损失函数和模型参数值。采用决策树作为基学习器的梯度提升法算法被称为 GBDT，GBDT 算法中损失函数的负梯度称为准残差 r 。GBDT 的基本思想是将每一步训练得到的准残差 r 最小化作为学习目标，之所以称为准残差，是为了强调普适性，因为如果使用的是二次损失函数，准残差 r 就是残差（响应变量实际值与拟合值之差，下面有证明），而对于其他一般损失函数来说，则是残差的近似值。

|| 梯度提升法 (Gradient Boosting machine)

具体实现过程是：

假设基学习器为 $f(x)$ ，由基学习器构成的强学习器为 $F(x)$ 。如果我们已经有了一个强学习器 $F_{n-1}(x)$ ，由 $n-1$ 个基学习器构成，那么即有：

$$F_{n-1}(x) = \sum_{i=1}^{n-1} f_i(x)$$

损失函数的负梯度 $-g_n(x)$ 即为：

$$-g_n(x) = -\frac{\partial L(y, (F_{n-1}(x)))}{\partial F_{n-1}(x)}$$

如果损失函数为二次损失函数，则有

$$L(y, (F_{n-1}(x))) = \frac{1}{2} \times (y - F_{n-1}(x))^2$$

将二次损失函数代入损失函数的负梯度，既有：

$$-g_n(x) = -\frac{\partial L(y, (F_{n-1}(x)))}{\partial F_{n-1}(x)} = -\frac{1}{2} \times \frac{\partial (y - F_{n-1}(x))^2}{\partial F_{n-1}(x)} = y - F_{n-1}(x) = r_{n-1}$$

求偏导

|| 梯度提升法 (Gradient Boosting machine)

所以对于二次损失函数来说，损失函数的负梯度（准残差）拟合的就是残差，而对于其他一般损失函数来说，拟合的是残差的近似值。

然后我们以该准残差最小化为目标构建下一个基学习器为 $f_n(x)$ ，则生成新的强学习器，把 $F_{n-1}(x)$ 更新成 $F_n(x)$ 。

在前述计算过程中，第一步以模型损失函数最小化为目标构建初始基学习器，得到 $F_0(x)$ ；然后第二步以及之后的各步均以上步产生的准残差最小化为目标来优化参数，即将准残差对特征变量进行回归，计算最优参数，并更新函数，或者说通过不断模型的迭代，来减小准残差，达到无限逼近目标值的效果。

|| 回归问题损失函数

上节所讲述的梯度提升法，本质上是通过损失函数负梯度（损失函数的导数的负数）的不断下降实现梯度提升（拟合或预测效果提升），所以其中的关键在于损失函数的选择，这从根源上决定了梯度提升的效率和效果。对于回归问题，常用的损失函数除了前面提及的平方损失函数之外，还有拉普拉斯损失函数（绝对损失函数）、胡贝尔损失函数和分位数损失函数。

回归问题损失函数

一、平方损失函数

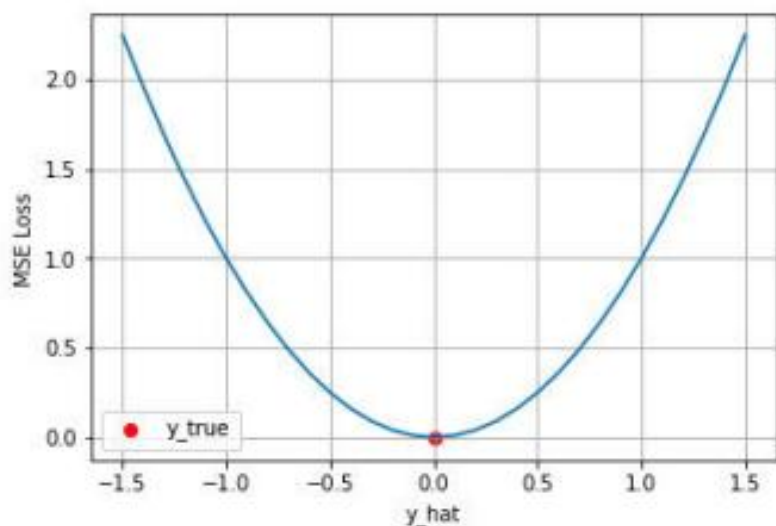
平方损失函数就是将损失函数设定为误差的平方,误差即响应变量实际值与拟合值之差。
数学公式如下:

$$L(y, f(x)) = (y - f(x))^2$$

将平方损失函数针对样本示例个数进行平均,即得到均方差 Mean Squared Error (MSE) 损失(也称 L2 Loss),如果样本个数为 N ,均方差损失数学公式为:

$$L(y, f(x)) = \frac{1}{N} \sum_{i=1}^N (y - f(x))^2$$

下图比较直观的展示了当响应变量 y 的实际值 (y_{true}) 为 0, 拟合值 (y_{hat}) 取值为 $[-1.5, 1.5]$ 时, 均方差损失随拟合值 (y_{hat}) 变化情况, 是一种抛物线性质的二次函数关系。



|| 回归问题损失函数

二、拉普拉斯损失函数（绝对损失函数）

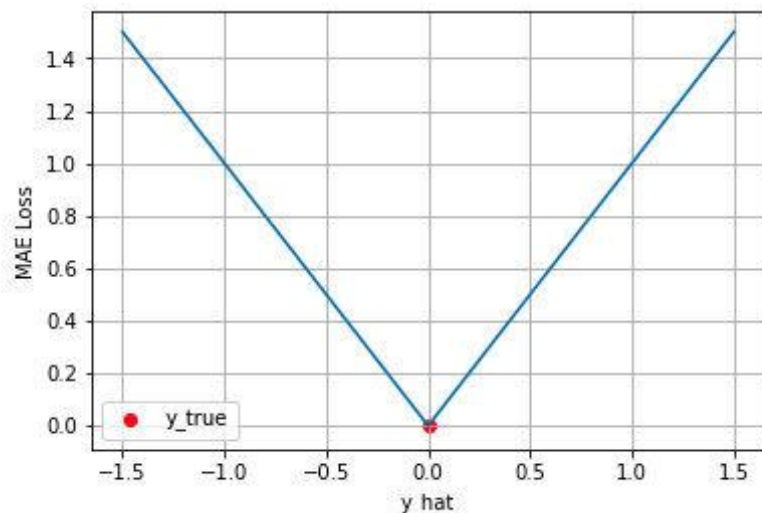
拉普拉斯损失函数（绝对损失函数）将损失函数设定为误差的绝对值，数学公式如下：

$$L(y, f(x)) = |y - f(x)|$$

在数据存在极端值的情形下，误差平方损失函数方法会因为误差平方的变换使得损失函数数值比较大，而拉普拉斯损失函数考虑的是误差的绝对值，所以相对平方变换受极端值的影响更小，从而结果更为稳健。将拉普拉斯损失函数针对样本示例个数进行平均，即得到平均绝对误差损失 Mean Absolute Error Loss（也称 L1 Loss），如果样本个数为 N ，平均绝对误差损失数学公式为：

$$L(y, f(x)) = \frac{1}{N} \sum_{i=1}^N |y - f(x)|$$

下图比较直观的展示了当响应变量 y 的实际值（ y_true ）为 0，拟合值（ y_hat ）取值为 $[-1.5, 1.5]$ 时，平均绝对误差损失随拟合值（ y_hat ）变化情况，是一种等量变化的线性关系。因为是线性关系，所以相对于平方损失函数下的二次函数关系，收敛速度相对较慢。



|| 回归问题损失函数

三、胡贝尔损失函数

胡贝尔损失函数是平方损失函数和绝对损失函数的综合，数学公式如下：

$$L(y, f(x)) = \begin{cases} \frac{1}{2} (y - f(x))^2 & |y - f(x)| \leq \delta \\ \delta |y - f(x)| - \frac{1}{2} \delta^2 & |y - f(x)| > \delta \end{cases}$$

胡贝尔损失函数结合了两种方法的优点，用户首先需要设定临界值 δ （在学习过程开始之前设置，可通过交叉验证确定）。当误差绝对值小于等于临界值 δ 时，采用平方损失函数；而当误差绝对值大于临界值 δ 时，采用绝对损失函数，以削弱极端值的影响；而且临界值 δ 可以进行根据具体研究内容灵活调节，使得该方法在应用方面比较灵活。可以发现当 $|y - f(x)| = \delta$ 时，上下两部分函数的取值一致，均为 $\frac{1}{2}\delta^2$ ，从而保证了该函数连续可导。

对应的损失函数的负梯度为：

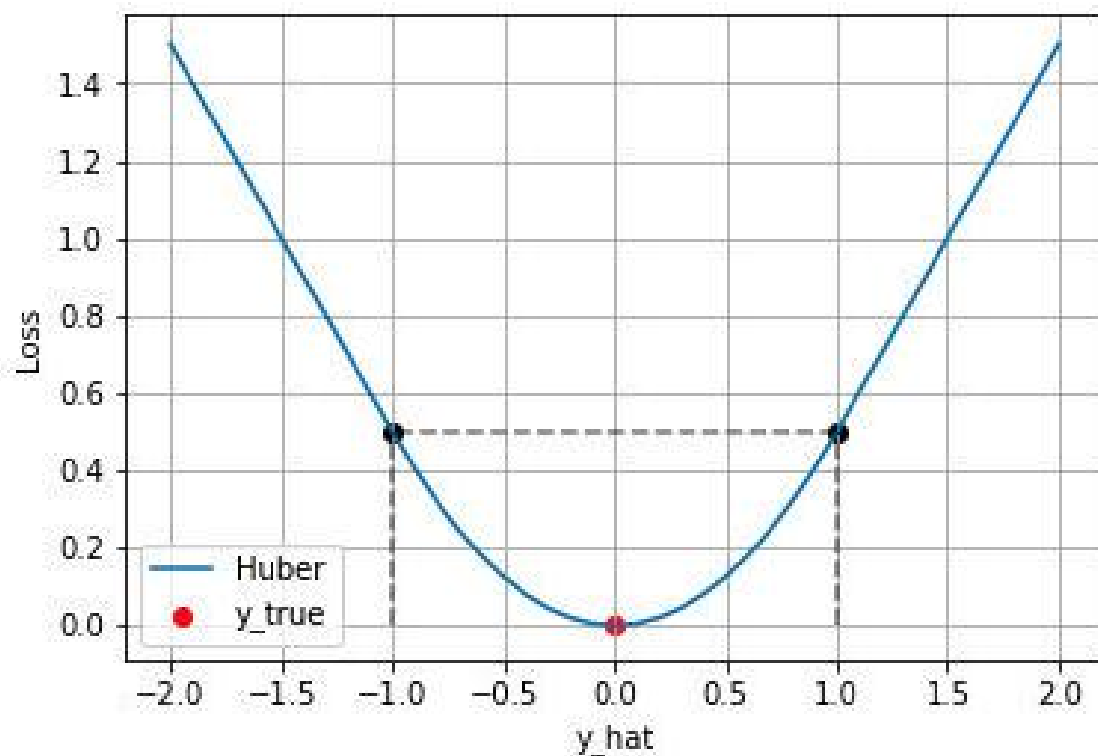
$$r(y_i, f(x_i)) = \begin{cases} y_i - f(x_i) & |y - f(x)| \leq \delta \\ \delta \text{sign}(y_i - f(x_i)) & |y - f(x)| > \delta \end{cases}$$

其中里面的 $\text{sign}(x)$ 函数为符号函数，其功能是取某个数的符号（正或负）：

当 $x > 0$ ， $\text{sign}(x) = 1$ ；当 $x = 0$ ， $\text{sign}(x) = 0$ ；当 $x < 0$ ， $\text{sign}(x) = -1$ 。

回归问题损失函数

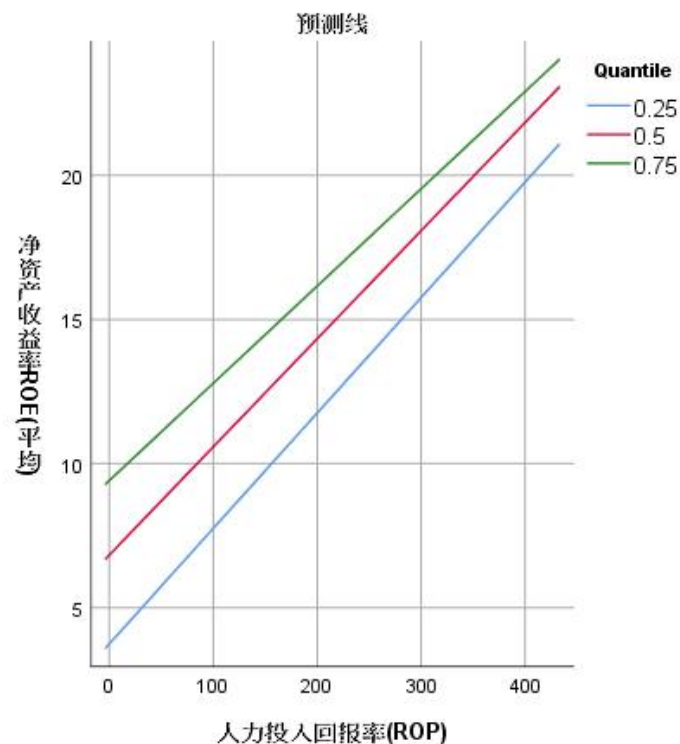
下图比较直观的展示了当临界值 δ 取值为 1，响应变量 y 的实际值 (y_{true}) 为 0，拟合值 (y_{hat}) 取值为 $[-2.0, 2.0]$ 时，胡贝尔损失随拟合值 (y_{hat}) 变化情况，可以发现拟合值 (y_{hat}) 取值为 $[-1, 1]$ 时，损失函数是一种均方差损失，呈现抛物线性质的二次函数关系；而当拟合值 (y_{hat}) 取值范围在 $[-1, 1]$ 之外时，损失函数是一种绝对损失，等量变化的线性关系。



回归问题损失函数

四、分位数损失函数

分位数损失函数用于分位数回归。分位数回归是定量建模的一种统计方法，最早由 Roger Koenker 和 Gilbert Bassett 于 1978 年提出，广泛应用于经济社会研究、医学保健等行业研究领域。分位数回归研究的是自变量与因变量的特定百分位数之间的关系。用更通俗易懂的语言来讲，就是普通线性回归的因变量与自变量的线性关系只有一个，包括斜率和截距；而分位数回归则根据自变量值处于的不同分位数值，分别生成对因变量的线性关系，可形成很多个回归方程。比如我们研究上市公司人力投入回报率对于净资产收益率的影响，当人力投入回报率处于较低水平时，其对净资产收益率的带动是较大的。但是当人力投入回报率达到较高水平时，其对净资产收益率的带动会减弱。也就是说，随着自变量值的变化，线性关系的斜率是会发生较大变动的，因此非常适合采用分位数回归方法。与普通线性回归相比，分位数回归对于目标变量的分布没有严格研究，也会趋向于抑制偏离观测值的影响，非常适合目标变量不服从正态分布、方差较大的情形。



回归问题损失函数

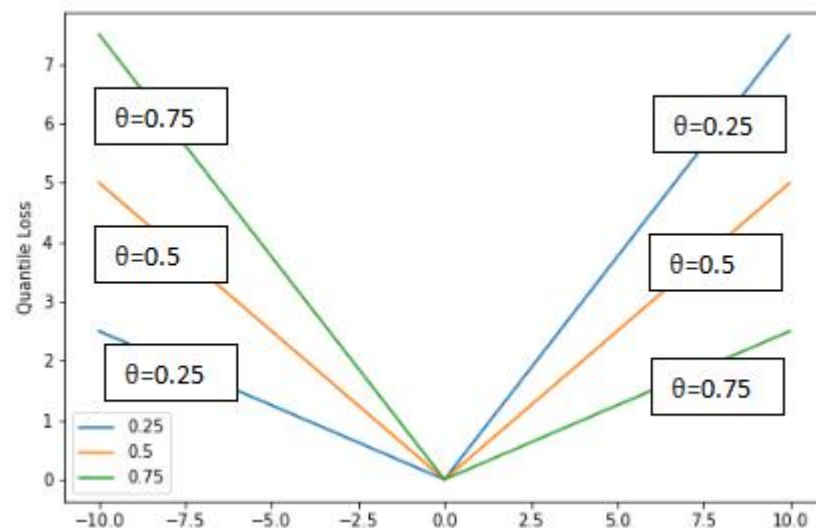
分位数损失函数的数学公式如下：

$$L(y, f(x)) = \sum_{y \geq f(x)} \theta |y - f(x)| + \sum_{y < f(x)} (1 - \theta) |y - f(x)|$$

其中 θ 为分位数，需要在学习过程开始之前指定。对应的损失函数的负梯度为：

$$r(y_i, f(x_i)) = \begin{cases} \theta & y_i \geq f(x_i) \\ 1 - \theta & y_i < f(x_i) \end{cases}$$

分位数损失分别用不同的 θ 控制高估和低估的损失，进而实现分位数回归。下图非常直观的展示了这一点：当分位数大于 0.5 时（图中为 0.75），说明样本比较靠前，那么低估的损失要比高估的损失更大；当分位数小于 0.5（图中为 0.25）时，说明样本比较靠后，高估的损失要比低估的损失更大；当分位数等于 0.5（图中为 0.25）时，即为中位数，分位数损失等同于平均绝对误差损失，从而起到了分位数回归优化的效果。



分类问题损失函数

对于分类问题，常用的损失函数除了前面提及的指数损失函数之外，还有逻辑损失函数、交叉熵损失函数。逻辑损失函数、交叉熵损失函数均是类似逻辑回归的对数似然函数的方法，其中逻辑损失函数用于二分类问题，交叉熵损失函数用于多分类问题。

分类问题损失函数

一、指数损失函数

$$L(y, f(x)) = e^{-yf(x)}$$

在使用指数损失函数时，梯度提升法等同于 Adaboost 算法。其中 y 的取值范围是 -1 或 1，由于在预测时是以 “ $f(x)$ ” 的符号来预测 y 取值是 1 还是 -1，所以 “ $yf(x) > 0$ ” 即为预测正确（实际为 1、预测 $f(x)$ 为正或实际为 -1、预测 $f(x)$ 为负），“ $yf(x) < 0$ ” 即为预测错误（实际为 -1、预测 $f(x)$ 为正或实际为 1、预测 $f(x)$ 为负）。

更一般的，“ $yf(x)$ ” 越大或者 “ $-yf(x)$ ” 越小，则预测越正确。我们把 “ $yf(x)$ ” 定义为裕度，“ $-yf(x)$ ” 即为负裕度，指数损失函数即为负裕度的指数函数，也就是说模型对于负裕度（预测错误）的惩罚是呈指数性增长的。如果数据质量不好（噪声过多或分类错误较多），该方法的稳健性较差，所以实务中不太好用。

分类问题损失函数

二、逻辑损失函数

逻辑损失函数用于二分类问题，本质上是类似逻辑回归的对数似然损失函数。损失函数的数学公式为：

$$L(y, f(x)) = \ln(1 + e^{-yf(x)})$$

其中 y 的取值范围是 -1 或 1 且服从 logit 模型分布，可以验证不论是取值为 -1 还是 1，条件概率都为：

$$p(y|x) = \frac{1}{1 + e^{-yf(x)}}$$

将条件概率代入损失函数，即有：

$$L(y, f(x)) = -\ln p(y|x)$$

也就是说，逻辑损失函数就是 logit 模型的对数似然函数的负数值。

对应的损失函数的负梯度为：

$$-g(x) = -\left\{ \frac{\partial L(y, f(x))}{\partial f(x)} \right\} = \frac{ye^{-yf(x)}}{1 + e^{-yf(x)}}$$

分类问题损失函数

三、交叉熵损失函数

交叉熵损失函数用于多分类问题，相对更复杂些，但本质上对应的是多元逻辑回归和二元逻辑回归的复杂度差别。假设类别数为 K ，损失函数公式为：

$$L(y, f(x)) = - \sum_{k=1}^K y_k \ln p_k(x)$$

其中 y_k 为虚拟变量， $p_k(x)$ 为 y 分类为 k 的条件概率。所以，前面介绍的逻辑损失函数事实上是交叉熵损失函数在响应变量为二分类时的特例，逻辑损失函数也称二值交叉熵损失函数。

综上所述，AdaBoost 通过加大分类错误样本示例的权重，使其在后面的训练中得到更多重视以实现效果提升，而梯度提升法则通过拟合先前基学习器的准残差，不断减小准残差实现模型的不断优化。相对于前面的 AdaBoost（自适应提升法），梯度提升法可以选择多种损失函数，在模型构建方面更加多元化，也更具普适性。

|| 随机梯度提升法

随机梯度提升法起源于梯度提升法，其基本思想还是梯度提升法，区别之处在于在每一轮训练提升时，不再使用全部样本示例，而是仅随机抽取一部分样本、基于抽取的样本示例子集拟合模型，然后计算准残差，将准残差对特征变量进行回归，计算最优参数，并更新函数，减小准残差，逼近目标值。需要注意的是，与袋装法那种有放回、保持样本容量不变、样本权重不变的均匀抽样不同，与AdaBoost那种根据错误率、从加权的分布中进行抽样也不同，此处的抽样是“无放回”抽样且样本权值均匀。这一抽样过程也不同于随机森林算法，随机森林算法除了像装袋法那样有放回、保持样本容量不变、样本权重不变的均匀抽样之外，每次抽样是还仅选取部分特征变量，而随机梯度提升法则在每轮训练中都包含了全部的特征变量。

|| 随机梯度提升法

相对于前面的梯度提升法，随机梯度提升法的优势体现在以下几个方面：

(1) 因为每轮训练不再使用全部样本示例，而是仅随机抽取一部分样本，所以一方面运行速度更快，另一方面学习速度更慢，更不容易出现过拟合。

(2) 因为是无放回的抽样，所以各个样本示例子集之间的相关性更弱，所以一方面更大程度的降低了模型的方差，另一方面更有助于模型在全局范围而非局部范围内寻找最优参数，优化了模型性能。

(3) 因为每次子抽样仅适用一部分样本，所以能够产生袋外观测值，计算袋外误差，评估模型性能。

不难看出，在随机梯度提升法中，非常重要的参数就是每次随机抽样的比例是多少，如果参数设置的过大，那么就无法起到随机抽样降低方差的作用，极端情形下如果将比例设置为100%，就会退化为一般的梯度提升法；而如果参数设置的过小，相当于仅适用少量信息来估计整体模型，那么在拟合模型方面就会出现较大的偏差，所以要找到最为合适的平衡点，这一平衡点也可以通过交叉验证的方式来确定。

||| XGBoost算法

XGBoost (Extreme Gradient Boosting) , 也称极限梯度提升算法, 为陈天奇所设计, 基础方法仍旧是梯度提升法, 但是做了很多改进, 包括使用牛顿法计算下降方向、改进决策树算法、使用稀疏矩阵等等, 被认为是在解决分类问题和回归问题上都拥有超高性能的先进评估器, 尤其比较适用于样本示例全集容量较大的数据, 目前已广泛应用于建模平台比赛、数据咨询等行业。



PART 02

数据准备

数据准备

本章我们用到的数据包括上一章中的“数据13.1”以及新数据“数据15.1”“数据15.2”。

“数据15.1”来源于万得资讯发布的，依据证监会行业分类的CSRC软件和信息技术服务业部分上市公司2019年末财务指标横截面数据。数据中的变量包括“市净率 (PB)”“净资产收益率 (ROE)”“资产负债率 (debt)”“总资产周转率 (assetturnover)”“研发费用同比增长 (rdgrow)”“投入资本回报率 (roic)”“人力投入回报率 (rop)”“经营活动净收益/利润总额 (netincomeprofit)”“保守速动比率 (quickratio)”“营业收入同比增长率 (incomegrow)”“净利润同比增长率 (netprofitgrow)”和“经营活动产生的现金流量净额同比增长率 (cashflowgrow)”12项。

针对“数据15.1”，我们以pb为响应变量，以roe、debt、assetturnover、rdgrow、roic、rop、netincomeprofit、quickratio、incomegrow、netprofitgrow、cashflowgrow为特征变量，使用回归提升法进行拟合。

数据准备

针对“数据13.1”（在第13章中已介绍），我们以是否发生违约（credit）为响应变量，以年龄（age）、受教育程度（education）、工作年限（workyears）、居住年限（resideyears）、年收入水平（income）、债务收入比（debtratio）、信用卡负债（creditdebt）、其他负债（otherdebt）为特征变量，使用二分类提升法进行拟合。

数据准备

“数据15.2”记录的是XX在线旅游服务提供商（虚拟名，如有雷同纯属巧合）注册会员客户的信息数据，具体包括grade、age、marry、years、income、education、consume、work、gender、family，分别用来表示客户的“会员等级”、“年龄”、“婚姻状况”、“会员注册年限”、“年收入水平”、“学历”、“月消费次数”、“工作性质”、“性别”、“家庭人数”等。Grade中1表示初级会员、2表示中级会员、3表示高级会员；marry中0表示未婚，1表示已婚；education中1表示初中及以下，2表示中专及高中，3表示本科及专科，4表示硕士研究生，5表示博士研究生；work中0表示有固定工作，1表示无固定工作；gender中1表示男，2表示女。

针对“数据15.2”，我们以grade为响应变量，以age、marry、years、income、education、consume、work、gender、family为特征变量，使用多分类提升法进行拟合。

载入分析所需要的模块和函数

在进行分析之前，我们首先载入分析所需要的模块和函数，读取数据集并进行观察。

示例

参阅教材内容



PART 03

回归提升法示例

|| 变量设置及数据处理

示例

参阅教材内容

线性回归算法观察

示例

参阅教材内容

|| 回归提升法（默认参数）

示例

参阅教材内容

|| 使用随机搜索寻求最优参数

示例

参阅教材内容

|| 绘制图形观察模型均方误差随基学习器数量变化情况

示例

参阅教材内容

|| 绘制图形观察模型拟合优度随基学习器数量变化情况

示例

参阅教材内容

|| 回归问题提升法特征变量重要性水平分析

示例

参阅教材内容

|| 绘制部分依赖图与个体条件期望图

示例

参阅教材内容

|| 最优模型拟合效果图形展示

示例

参阅教材内容

||| XGBoost回归提升法

示例

参阅教材内容



PART 04

二分类提升法示例

|| 变量设置及数据处理

示例

参阅教材内容

示例

参阅教材内容

|| 二分类提升法（默认参数）

示例

参阅教材内容

|| 使用随机搜索寻求最优参数

示例

参阅教材内容

|| 二分类问题提升法特征变量重要性水平分析

示例

参阅教材内容

|| 绘制部分依赖图与个体条件期望图

示例

参阅教材内容

模型性能评价

示例

参阅教材内容

|| 绘制ROC曲线

示例

参阅教材内容

|| 运用两个特征变量绘制二分类提升法决策边界图

示例

参阅教材内容

XGBoost二分类提升法

示例

参阅教材内容



PART 05

多分类提升法示例

|| 变量设置及数据处理

示例

参阅教材内容

多元Logistic回归算法观察

示例

参阅教材内容

多分类提升法（默认参数）

示例

参阅教材内容

|| 使用随机搜索寻求最优参数

示例

参阅教材内容

多分类问题提升法特征变量重要性水平分析

示例

参阅教材内容

|| 绘制部分依赖图与个体条件期望图

示例

参阅教材内容

模型性能评价

示例

参阅教材内容

XGBoost多分类提升法

示例

参阅教材内容



PART 06

习 题

习题

使用“数据5.1”数据文件（详情已在第5章中介绍），把响应变量设定为“V1征信违约记录”，将其他变量作为特征变量，具体包括“V2资产负债率”、“V6主营业务收入”、“V7利息保障倍数”、“V15银行负债”、“V9其他渠道负债”，构建二分类提升法模型。

- (1) 变量设置及数据处理
- (2) AdaBoost算法
- (3) 二分类提升法（默认参数）
- (4) 使用随机搜索寻求最优参数

- (5) 二分类问题提升法特征变量重要性水平分析
- (6) 绘制部分依赖图与个体条件期望图
- (7) 模型性能评价
- (8) 绘制ROC曲线
- (9) 运用两个特征变量绘制二分类提升法决策边界图
- (10) XGBoost二分类提升法